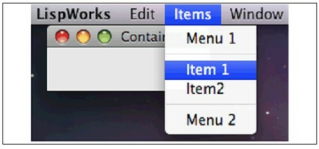

Figure 1: Menu with menu components

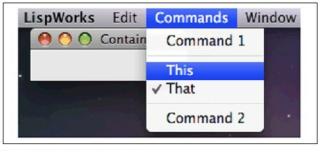

Figure 2: Radio buttons in the menu

which is a library for implementing portable window-based application interfaces. CAPI is a conceptually simple, CLOS-based model of interface elements and their interaction. It provides a standard set of these elements and their behaviours, as well as giving you the opportunity to define elements of your own. CAPI currently runs under the X Window System with either GTK+ or Motif, and on Microsoft Windows and Mac OS X. Using CAPI with Motif is deprecated.

Let's create a few GUI elements using LispWorks and CAPI.

Menus

You can create menus for an application using the *menu* class. Let us start by creating a test-callback and a *hello* function, which we'll need to create and test our GUIs.

```
(defun test-callback (data interface)
  (display-message "Data ~S in interface ~S"
                   data interface))

(defun hello (data interface)
  (declare (ignore data interface))
  (display-message "Hello World"))
```

The following code then creates a CAPI interface with a menu, *Foo*, which contains four items. Choosing any of these items displays its arguments. Each item has the callback specified by the *:callback* keyword.

```
(make-instance 'menu
    :title "Foo"
    :items '("One" "Two" "Three" "Four")
    :callback 'test-callback)

(make-instance 'interface
    :menu-bar-items (list *))

(display *)
```

A sub-menu can be created simply by specifying a menu as one of the items of the top-level menu.

```
(make-instance 'menu
    :title "Bar"
    :items '("One" "Two" "Three" "Four")
    :callback 'test-callback)

(make-instance 'menu
    :title "Baz"
    :items (list 1 2 * 4 5)
    :callback 'test-callback)

(contain *)
```

This creates an interface that has a menu called *Baz*, which itself contains five items. The third item is another menu, *Bar*, which contains four items. Once again, selecting any item returns its arguments. Menus can be nested as deeply as required, using this method.

The *menu-component* class lets you group related *items* together in a menu. This allows similar menu items to share properties such as callbacks, and to be visually separated from other items in the menus. Menu components are actually choices.

Here is a simple example of a menu component. This creates a menu called *Items*, which has four items. *Menu 1* and *Menu 2* are ordinary menu items, but *Item 1* and *Item 2* are created from a menu component, and are therefore grouped together in the menu, as shown in Figure 1:

```
(setq component (make-instance 'menu-component
                        :items '("item 1" "item2")
                        :print-function 'string-capitalize
                        :callback 'test-callback))

(contain (make-instance 'menu
                        :title "Items"
                        :items
                        (list "menu 1" component "menu 2")
                        :print-function 'string-capitalize
                        :callback 'hello)
         :width 150
         :height 0)
```

Radio buttons

Menu components allow you to specify, via the *:interaction*